

💎 💎 所属专栏: [Linux](#) 💎 💎

💎 💎 作者主页: [嵌某](#) 💎 💎

ELF文件

什么是编译？编译就是将程序源代码编译成能让CPU直接执行的机器代码

如果我们要编译一个 `.c` 文件，使用 `gcc -c` 将 `.c` 文件编译为二进制文件 `.o`，如果一个项目有多个 `.c` 文件，会生成多个 `.o` 文件，如果我们修改一个 `.c` 文件，那么只需要将这一个 `.c` 文件重新编译即可。不需要浪费时间去重新编译整个工程文件。目标文件就是一种二进制文件，`ELF` 是二进制文件的格式，也是对二进制文件的封装

```
ubuntu@VM-4-4-ubuntu:~/Code/25/2_13$ file test.o
test.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

 1740388705932

要理解编译的细节，就先要了解 `ELF` 文件，以下四种都是 `ELF` 文件：

1. **可重定位文件 (Relocatable File)**：即 `xxx.o` 文件。包含适合于与其他目标文件链接来创建可执行文件或者共享目标文件的代码和数据。
2. **可执行文件 (Executable File)**：即可执行程序。
3. **共享目标文件 (Shared Object File)**：即 `xxx.so` 文件。
4. **内核转储 (core dumps)**，存放当前进程的执行上下文，用于 `dump` 信号触发。

每个 `ELF` 文件都由以下部分组成：

1. **ELF头 (ELF header)**：描述文件的主要特性。其位于文件的开始位置，它的主要目的是定位文件的其他部分。
2. **程序头表 (Program header table)**：列举了所有有效的段 (`segments`) 和他们的属性。表里记着每个段的开始的位置和位移 (`offset`)、长度，毕竟这些段，都是紧密的放在二进制文件中，需要段表的描述信息，才能把他们每个段分割开。
3. **节头表 (Section header table)**：包含对节 (`sections`) 的描述。
4. **节 (Section)**：`ELF` 文件中的基本组成单位，包含了特定类型的数据。`ELF` 文件的各种信息和数据都存储在不同的节中，如代码节存储了可执行代码，数据节存储了全局变量和静态数据等。

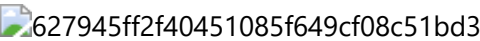
最常见的节有代码节 `.text` 和数据节 `.data`，代码节用于保存机器指令，是程序的主要执行部分。数据节保存已经初始化的全局变量和局部静态变量。

 7f4d909663164a7f8439356d841446c2

ELF形成到加载的大概轮廓

ELF形成可执行文件

- 1. 将多个C/C++源代码翻译为目标 .o 文件
- 2. 将多个.o文件的section合并



具体的合并方式会比这复杂，是在链接时合并的，并且还涉及到对库的合并

ELF可执行文件加载

- 在一个ELF文件中也会有许多不同的Section，在加载到内存的时候，也会进行Section的合并，形成Segment
- 合并原则：相同属性（可读，可写，可执行，加载时需要申请内存空间）
- 即使不同的Section，加载到内存之后，可能都会在一个Segment里
- 很显然，这种合并方法已经在形成ELF时就已经确定了，并且记录在了ELF的程序头表Program header table中

```
# 查看可执行程序的Section
ubuntu@VM-4-4-ubuntu:~/Code/25/2_13$ readelf -S a.out
There are 31 section headers, starting at offset 0x37b8:

Section Headers:
  [Nr] Name                Type              Address            Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                      NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0   0   0
  [ 1] .interp                PROGBITS          0000000000000318  00000318
       000000000000001c  0000000000000000   A     0   0   1
  [ 2] .note.gnu.pr[...]     NOTE             0000000000000338  00000338
       0000000000000030  0000000000000000   A     0   0   8
  [ 3] .note.gnu.bu[...]     NOTE             0000000000000368  00000368
       0000000000000024  0000000000000000   A     0   0   4
  [ 4] .note.ABI-tag          NOTE             000000000000038c  0000038c
       0000000000000020  0000000000000000   A     0   0   4
  [ 5] .gnu.hash              GNU_HASH          00000000000003b0  000003b0
       0000000000000024  0000000000000000   A     6   0   8
  [ 6] .dynsym                DYNSYM           00000000000003d8  000003d8
       00000000000000c0  0000000000000018   A     7   1   8
  [ 7] .dynstr                STRTAB           0000000000000498  00000498
       00000000000000c8  0000000000000000   A     0   0   1
  [ 8] .gnu.version           VERSYM           0000000000000560  00000560
       0000000000000010  0000000000000002   A     6   0   2
  [ 9] .gnu.version_r         VERNEED          0000000000000570  00000570
       0000000000000050  0000000000000000   A     7   2   8
 [10] .rela.dyn              RELA             00000000000005c0  000005c0
       00000000000000c0  0000000000000018   A     6   0   8
 [11] .rela.plt              RELA             0000000000000680  00000680
       0000000000000018  0000000000000018  AI     6  24   8
 [12] .init                  PROGBITS          0000000000001000  00001000
       000000000000001b  0000000000000000  AX     0   0   4
 [13] .plt                   PROGBITS          0000000000001020  00001020
```

	0000000000000020	0000000000000010	AX	0	0	16
[14]	.plt.got	PROGBITS	0000000000001040	00001040		
	0000000000000010	0000000000000010	AX	0	0	16
[15]	.plt.sec	PROGBITS	0000000000001050	00001050		
	0000000000000010	0000000000000010	AX	0	0	16
[16]	.text	PROGBITS	0000000000001060	00001060		
	0000000000000107	0000000000000000	AX	0	0	16
[17]	.fini	PROGBITS	0000000000001168	00001168		
	000000000000000d	0000000000000000	AX	0	0	4
[18]	.rodata	PROGBITS	0000000000002000	00002000		
	0000000000000014	0000000000000000	A	0	0	4
[19]	.eh_frame_hdr	PROGBITS	0000000000002014	00002014		
	0000000000000034	0000000000000000	A	0	0	4
[20]	.eh_frame	PROGBITS	0000000000002048	00002048		
	00000000000000ac	0000000000000000	A	0	0	8
[21]	.init_array	INIT_ARRAY	0000000000003da8	00002da8		
	0000000000000008	0000000000000008	WA	0	0	8
[22]	.fini_array	FINI_ARRAY	0000000000003db0	00002db0		
	0000000000000008	0000000000000008	WA	0	0	8
[23]	.dynamic	DYNAMIC	0000000000003db8	00002db8		
	0000000000000200	0000000000000010	WA	7	0	8
[24]	.got	PROGBITS	0000000000003fb8	00002fb8		
	0000000000000048	0000000000000008	WA	0	0	8
[25]	.data	PROGBITS	0000000000004000	00003000		
	0000000000000010	0000000000000000	WA	0	0	8
[26]	.bss	NOBITS	0000000000004010	00003010		
	0000000000000008	0000000000000000	WA	0	0	1
[27]	.comment	PROGBITS	0000000000000000	00003010		
	0000000000000026	0000000000000001	MS	0	0	1
[28]	.symtab	SYMTAB	0000000000000000	00003038		
	000000000000003c0	0000000000000018		29	21	8
[29]	.strtab	STRTAB	0000000000000000	000033f8		
	00000000000002a0	0000000000000000		0	0	1
[30]	.shstrtab	STRTAB	0000000000000000	00003698		
	000000000000011a	0000000000000000		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), l (large), p (processor specific)

查看Section合并的Segment
readelf -l a.out

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x10a0
There are 13 program headers, starting at offset 64

Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags	Align
PHDR	0x0000000000000040	0x0000000000000040	0x0000000000000040	
	0x00000000000002d8	0x00000000000002d8	R	0x8
INTERP	0x0000000000000318	0x0000000000000318	0x0000000000000318	

```

                                0x000000000000001c 0x000000000000001c R      0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD      0x0000000000000000 0x0000000000000000 0x0000000000000000
          0x00000000000000698 0x00000000000000698 R      0x1000
LOAD      0x00000000000001000 0x00000000000001000 0x00000000000001000
          0x00000000000001ed 0x00000000000001ed R E    0x1000
LOAD      0x00000000000002000 0x00000000000002000 0x00000000000002000
          0x00000000000000fc 0x00000000000000fc R      0x1000
LOAD      0x00000000000002da8 0x00000000000003da8 0x00000000000003da8
          0x0000000000000268 0x0000000000000270 RW     0x1000
DYNAMIC   0x00000000000002db8 0x00000000000003db8 0x00000000000003db8
          0x00000000000001f0 0x00000000000001f0 RW     0x8
NOTE      0x0000000000000338 0x0000000000000338 0x0000000000000338
          0x0000000000000030 0x0000000000000030 R      0x8
NOTE      0x0000000000000368 0x0000000000000368 0x0000000000000368
          0x0000000000000044 0x0000000000000044 R      0x4
GNU_PROPERTY 0x0000000000000338 0x0000000000000338 0x0000000000000338
          0x0000000000000030 0x0000000000000030 R      0x8
GNU_EH_FRAME 0x0000000000000201c 0x0000000000000201c 0x0000000000000201c
          0x0000000000000034 0x0000000000000034 R      0x4
GNU_STACK 0x0000000000000000 0x0000000000000000 0x0000000000000000
          0x0000000000000000 0x0000000000000000 RW     0x10
GNU_RELRO 0x00000000000002da8 0x00000000000003da8 0x00000000000003da8
          0x0000000000000258 0x0000000000000258 R      0x1

Section to Segment mapping:
Segment Sections...
00
01      .interp
02      .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .gnu.hash
.dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt
03      .init .plt .plt.got .plt.sec .text .fini
04      .rodata .eh_frame_hdr .eh_frame
05      .init_array .fini_array .dynamic .got .data .bss
06      .dynamic
07      .note.gnu.property
08      .note.gnu.build-id .note.ABI-tag
09      .note.gnu.property
10      .eh_frame_hdr
11
12      .init_array .fini_array .dynamic .got

```

为什么要将Section合并成为Segment？

- Section合并的主要原因是为了减少页面碎片，提高内存使用效率。如果不进行合并，假设页面大小为4096字节（内存块基本大小，加载，管理的基本单位），如果.text部分为4097字节，.init部分为512字节，那么它们将占用3个页面，而合并后，它们只需2个页面。
- 此外，操作系统在加载程序时，会将具有相同属性的section合并成一个大的segment，这样就可以实现不同的访问权限，从而优化内存管理和权限访问控制

对于程序头表和节头表又有什么用呢，其实 ELF 文件提供 2 个不同的视图/视角来让我们理解这两个部分：

- 链接视图 Linking view 对应节头表 Section header table

- 文件结构的粒度更细，将文件按功能模块的差异进行划分，静态链接分析的时候一般关注的是链接视图，能够理解 ELF 文件中包含的各个部分的信息。
- 为了空间布局上的效率，将来在链接目标文件时，链接器会把很多节 section 合并规整成可执行的段 segment、可读写的段、只读段等。合并之后，空间利用率提高了。否则很小很小的一段，会浪费很多物理内存（物理内存页分配一般都是 4k 的整数倍一起给你），所以，链接器趁着链接就把小块们都合并了。

- 执行视图 execution view 对应程序头表 Program header table

- 告诉操作系统，如何加载可执行文件，完成进程内存的初始化。一个可执行程序的文件格式中，一定有 program header table。

说白了，一个在链接时作用，一个在运行加载中作用。

 2b29a68abd4a4073ad929508727071c3

从链接视图来看：

- 命令 `readelf -S hello.o` 可以帮助查看 ELF 文件的节头表
- `.text` 节：是保存了程序代码指令的代码节。
- `.data` 节：保存了初始化的全局变量和局部静态变量等数据。
- `.rodata` 节：保存了只读的数据如一行 C 语言代码中的字符串。由于 `.rodata` 节是只读的，所以只能存在于一个可执行文件的只读段中。因此，只能是在 `text` 段（不是 `data` 段）中找到 `.rodata` 节
- `.BSS` 节：为未初始化的全局变量和局部静态变量预留位置
- `.symtab` 节：Symbol Table 符号表，就是源码中的函数名，变量名和代码的对应关系。
- `.got.plt` 节（全局偏移表-过程链接表）：`.got` 节保存了全局偏移表。`.got` 节和 `.plt` 节一起提供了对导入的共享函数的访问入口，由动态链接器在运行时进行修改。

- 使用 `readelf` 命令查看 `.so` 文件可以看到该节

从执行视图来看：

- 告诉操作系统哪些模块可以被加载进内存。
- 加载进内存之后哪些分段是可读可写，哪些分段是只读，哪些分段是可执行的

我们可以在 ELF 头中找到文件的基本信息，以及可以看到 ELF 头是如何定位程序头表和节头表的。例如：

```
// 查看目标文件
$ readelf -h hello.o
ELF Header:
Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
Class: ELF64 # 文件类型
Data: 2's complement, little endian # 指定的编码方式
Version: 1 (current)
OS/ABI: UNIX - System V
ABI Version: 0
Type: REL (Relocatable file) # 指出ELF文件的类型
```

```

Machine: Advanced Micro Devices X86-64 # 该程序需要的体系结构
Version: 0x1
Entry point address: 0x0 # 系统第一个传输控制的虚拟地址，在那启动进程。假如文件没有如何
关联的入口点，该成员就保持为0。
Start of program headers: 0 (bytes into file)
Start of section headers: 728 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes) # 保存着ELF头大小(以字节计数)
Size of program headers: 0 (bytes) # 保存着在文件的程序头表 (program header table)
中一个入口的大小
Number of program headers: 0 # 保存着在程序头表中入口的个数。因此，e_phentsize和
e_phnum的乘积就是表的大小(以字节计数)。假如没有程序头表，变量为0。
Size of section headers: 64 (bytes) # 保存着section头的大小(以字节计数)。一个section
头是在section头表的一个入口
Number of section headers: 13 # 保存着在section headertable中的入口数目。因此，
e_shentsize和e_shnum的乘积就是section头表的大小(以字节计数)。假如文件没有section头表，
值为0。
Section header string table index: 12 # 保存着跟section名字字符表相关入口的section头
表(section header table)索引。

// 查看可执行程序
$ gcc *.o
$ readelf -h a.out
ELF Header:
Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
Class: ELF64
Data: 2's complement, little endian
Version: 1 (current)
OS/ABI: UNIX - System V
ABI Version: 0
Type: DYN (Shared object file)
Machine: Advanced Micro Devices X86-64
Version: 0x1
Entry point address: 0x1060
Start of program headers: 64 (bytes into file)
Start of section headers: 14768 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 13
Size of section headers: 64 (bytes)
Number of section headers: 31
Section header string table index: 30

```

理解连接与加载

静态连接

无论是自己的.o还是静态库中的.o，其本质都是将.o文件与本地源文件进行连接，所以研究静态链接，本质就是研究这个。

使用objdump -d 将.o文件反汇编，查看code.o和hello.o

```
ubuntu@VM-4-4-ubuntu:~/Code/25/2_24$ objdump -d code.o

code.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <run>:
   0:  f3 0f 1e fa          endbr64
   4:  55                  push   %rbp
   5:  48 89 e5            mov    %rsp,%rbp
   8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <run+0xf>
   f:  48 89 c7            mov    %rax,%rdi
  12:  e8 00 00 00 00      call   17 <run+0x17>
  17:  90                  nop
  18:  5d                  pop    %rbp
  19:  c3                  ret

ubuntu@VM-4-4-ubuntu:~/Code/25/2_24$ objdump -d hello.o

hello.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
   0:  f3 0f 1e fa          endbr64
   4:  55                  push   %rbp
   5:  48 89 e5            mov    %rsp,%rbp
   8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
   f:  48 89 c7            mov    %rax,%rdi
  12:  e8 00 00 00 00      call   17 <main+0x17>
  17:  b8 00 00 00 00      mov    $0x0,%eax
  1c:  e8 00 00 00 00      call   21 <main+0x21>
  21:  b8 00 00 00 00      mov    $0x0,%eax
  26:  5d                  pop    %rbp
  27:  c3                  ret
```

可以很清楚的看到在code.o中call中的地址全是0，这说明在code.c中根本不认识printf函数，而在hello.o中也不认识printf和run。因为它们的跳转地址都变成了全零。

因为在编译的时候，编译器是完全不知道其他函数是什么，位于内存的哪个区块，代码长什么样，所以编译器只能将这些函数的跳转地址都先设为0。这个地址会在链接的时候被修正，为了让链接器在链接时能正确的重定位到这些被修正的地址，在代码块.data中存在一张重定位表，在链接的时候就会根据这张表对地址进行修正。

静态链接就是将本地的.o文件和库里的.a文件进行链接，链接其实就是将编译之后的所有目标文件连同用到的一些静态库运行时库组合，拼装成一个独立的可执行文件。其中就包括我们之前提到的地址修正，当所有模块

组合在一起之后，链接器会根据我们的.o文件或者静态库中的重定位表找到那些需要被重定位的函数全局变量，从而修正它们的地址。这其实就是静态链接的过程。

image-20250226131742897

所以，链接过程中会涉及到对.o中外部符号进行地址重定位。

ELF加载与进程地址空间

虚拟地址/逻辑地址

- 一个ELF程序，在没有加载到内存的时候，有没有地址？
- 进程mm_struct、vm_area_struct在进程刚刚创建的时候，初始化数据从哪来的？

ans:

一个ELF程序，在没有加载到内存的时候，是有地址的，**当代计算机工作的时候都采用平坦模式进行工作**。所以也要求ELF对自己的代码和数据进行统一编址，我们再看hello.o的反汇编，最左侧的就是ELF的虚拟地址，严格来说应该叫做逻辑地址（起始地址+偏移量）但是我们认为起始地址就是0。

也就是说其实虚拟地址在程序还没有加载到内存的时候，就已经把可执行程序进行统一编址了。

```
ubuntu@VM-4-4-ubuntu:~/Code/25/2_24$ objdump -S hello.o

hello.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
   0:  f3 0f 1e fa      endbr64
   4:  55              push    %rbp
   5:  48 89 e5        mov     %rsp,%rbp
   8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
   f:  48 89 c7        mov     %rax,%rdi
  12:  e8 00 00 00 00  call   17 <main+0x17>
  17:  b8 00 00 00 00  mov     $0x0,%eax
  1c:  e8 00 00 00 00  call   21 <main+0x21>
  21:  b8 00 00 00 00  mov     $0x0,%eax
  26:  5d              pop     %rbp
  27:  c3              ret
```

进程mm_struct、vm_area_struct在进程创建的时候，从ELF文件的各个segment获取，每一个segment有自己的其实地址和长度，就可以填充内核中的strat、end、页表等数据。

所以操作系统和编译器都支持虚拟地址机制。

所以ELF文件在被编译好之后，会在ELF header的Entry字段中记录程序的入口地址，运行时，这个地址最先被加载到CPU寄存器里面，然后跟着运行后面的代码。


```
ubuntu@VM-4-4-ubuntu:~/Code/25/2_24$ gcc -o hello hello.c
ubuntu@VM-4-4-ubuntu:~/Code/25/2_24$ readelf -h hello
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1060
  Start of program headers:              64 (bytes into file)
  Start of section headers:              13968 (bytes into file)
  Flags:                                 0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              13
  Size of section headers:               64 (bytes)
  Number of section headers:              31
  Section header string table index: 30
```

所以静态链接就是将本地ELF文件.o和库中的ELF文件.a链接，通过program header table的信息将他们的segment合并，统一进行平坦编址、这个是逻辑地址。然后程序运行的时候，内核生成对应的task_struct，ELF可执行文件加载到内存中，拥有了对应的物理地址，将虚拟地址和物理地址分别填入页表，对应的mm_struct等信息一填，然后将程序的Entry point address加载到CPU中，程序开始执行。

image-20250226203350306

动态链接与动态库加载

首先，动态链接比静态链接要常用的多，大部分官方库都是默认采用动态库。使用ldd命令查看一个可执行程序依赖的各种库

```
ubuntu@VM-4-4-ubuntu:~/Code/25/2_24$ ldd hello
linux-vdso.so.1 (0x00007ffddc939000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f5174a00000)
/lib64/ld-linux-x86-64.so.2 (0x00007f5174e10000)
```

这里的libc.so是C语言的运行时库里面提供了常用的标准输入输出文件字符串处理等功能。

这里要说的是，静态链接会将编译产生的所有目标文件，连同到用到的各种的库一起合并为一个独立的可执行文件，虽然它不需要任何额外依赖就可以运行。但是其最大的问题就是生成的文件体积太大了，相当耗费内存资源。随着软件复杂度的提升，操作系统也越来越臃肿，不同的软件可能都包含了相同的部分功能和代码，这显然不利于计算机的发展。

所以，动态链接的优势就体现在这里，我们将共享的代码提取出来，封装为一个单独的动态链接库。等到程序运行的时候加载到内存，不但可以节省空间，而且同一个模块只需要在内存中加载一次，就可以被不同的进程所共享。

那么，动态链接是如何工作的？

首先，结论、动态链接将链接的整个过程推迟到了程序加载的时候。我们运行一个程序，操作系统首先将程序的数据代码连同它用到的一系列动态库先加载到内存，其中每个动态库的加载地址都是不固定的从，操作系统会根据当前的内存使用情况为它们动态分配一段内存。当动态库被加载到内存后，一旦物理地址（内存里的地址）确定，我们就可以去修正动态库中的那些函数的跳转地址了。

要彻底理清楚，先要知道我们的程序是怎么开始启动的

在C/C++程序中，当程序开始运行时，它不是从main函数开始执行的。实际上，程序的入口点是_start这是一个由C运行时库glibc或链接器ld提供的特殊函数。在_start函数中，会执行一系列初始化操作，这些操作包括：

- 设置堆栈，为程序创建一个初始堆栈环境。
- 初始化数据段，将程序的数据段（如全局和静态变量）从初始化数据段复制到对应的内存位置，并清零未初始化的数据段
- 动态链接，_start函数会调用动态链接器的代码来解析和加载程序所依赖的动态库shared libraries。动态链接器会处理所有的符号解析和重定位，确保程序中的函数调用和变量访问能够正确的映射到动态库的实际地址。

动态链接器：（以下简称Dynamic Linker）

- 动态链接器（如ld-linux.so）负责在程序运行时加载动态库
- 当程序启动时，Dy Linker会解析程序中的动态库依赖，并加载这些库到内存中。

环境变量和配置文件：

- Linux系统通过环境变量和配置文件来指定动态库的默认搜索路径。
- 这些路径会被动态链接器在加载动态库时搜索。

缓存文件：

- 为了提高动态库的加载效率，Linux系统会维护一个名为/etc/ld.so.cache的缓存文件。
- 该文件包含了系统中所有已知的动态库路径和相关信息，动态链接器在加载动态库时会首先搜索这个缓存文件。

- 调用__libc_start_main，一旦动态链接完成，_start函数就会被调用。__libc_start_main这个函数又glibc提供，负责执行一些额外的初始化工作，比如设置信号处理函数，初始化线程库（如果使用了线程）等。
- 调用main函数，最后__libc_start_main会调用mian函数，此时，程序的控制权才正式的交给了用户编写的代码。
- 处理main函数的返回值，main函数返回时__libc_start_main负责处理这个返回值，并最终调用_exit函数来终止程序。

上述过程描述的时C/C++程序在执行main函数之前的一系列操作，这部分被编译器“优化”掉的操作，对于大多数程序员来说时透明的。程序员通常只需要关注main函数中的代码，而不需要关心程序底层的初始化过程。了解这些过程可以使我们更好的理解程序的执行流程和Dbg。

动态库中的相对地址

动态库为了能随时加载，支持并映射到任意进程的任意位置，对动态库中的方法统一编制，采用相对编址（平坦模式）的方案。（可执行程序，静态库，动态库都采用从全零开始的相对编址）

动态库本质也是一个磁盘上的文件，要加载也是要被打开的，要让进程看到动态库，不仅要加载到内存中，进程要访问动态库，还要让进程看到动态库，也就是将动态库的虚拟地址和物理地址分别填入进程的页表中

 image-20250227143654849

通过上面这张图，动态库的加载过程我们清楚了，那么进程是怎么进行库调用的呢？

- 库已经被我们映射到了当前进程的地址空间中
- 库的起始地址我们知道了，库中每一个方法的偏移量我们也知道。
- 所以库的起始地址+方法偏移量就可以定位库中的任意方法。
- 整个调用过程，从代码区跳转到共享区，调用完后返回代码区，整个过程都是在进程地址空间完成的。

 image-20250227145350745

- 也就是说程序运行之前，先把所有库加载并映射，所有库的起始虚拟地址我们都提前知道了。
- 然后对我们加载到内存中的程序的库函数调用进行地址修改，在内存中二次完成地址设置（加载地址重定位）
- Wait!!! 刚刚我们是不是修改了什么，修改了call后面的地址是吧？这不是代码段吗？代码段不是只读的不能修改吗？

全局偏移量表GOT (global offset table)

所以，动态链接的做法是在.data（或者库里面）预留一片区域用来存放函数的跳转地址，也被叫做全局偏移量表GOT，表中每一项都是本运行模块也要引用的一个全局变量或函数的地址。

.data区是可读写的，支持动态修改。

```
$ readelf -S a.out
...
[24] .got          PROGBITS          00000000000003fb8 00002fb8
      0000000000000048 0000000000000008 WA          0          0          8
...
$ readelf -l a.out # .got在加载的时候，会和.data合并成为一个segment，然后加载在一起
```

 image-20250227150827780

1. 由于代码段只读，我们不能直接修改代码段。有了GOT表，代码可以被所有进程共享。但在不同进程的地址空间中，各个动态库的绝对地址、相对位置都不同。反应到GOT表上，就是每个进程的每个动态库都有独立的GOT表，所以进程间不能共享GOT表
2. 在单个.so下，由于GOT表与.text的相对位置都是固定的，我们完全可以利用CPU的相对寻址找到GOT表
3. 在调用函数的时候会首先查表，然后根据表中的地址来进行跳转，这些地址在动态库加载时会被修改为真正的地址

4. 这种方式实现的动态链接就被叫做**PIC 地址无关代码**。也就是说，动态库不需要做任何修改，被加载到任意内存地址都能正常运行，并且被所有进程共享，这也是为什么在制作动态库时给编译器指定 **-fPIC** 参数的原因， $PIC = 相对编址 + GOT$

PLT 的作用和原理

由于动态链接在程序加载的时候需要对大量函数进行重定位，这一步显然是非常耗时的。为了进一步降低开销，我们的操作系统还做了一些其他的优化，比如**延迟绑定** **Lazy Binding**，或者也叫**PLT**（过程连接表 **Procedure Linkage Table**）。与其在程序一开始就对所有函数进行重定位，不如将这个过程推迟到函数第一次被调用的时候，因为绝大多数动态库中的函数可能在程序运行期间一次都不会被使用到。

思路是：**GOT**中的跳转地址默认会指向一段辅助代码，它也被叫做桩代码/**stub**。在我们第一次调用函数的时候，这段代码会负责查询真正函数的跳转地址，并且去更新**GOT**表。于是我们再次调用函数的时候，就会直接跳转到动态库中真正的函数实现。

总而言之，动态链接实际上将链接的整个过程，比如符号查询、地址的重定位从编译时推迟到了程序的运行时，它虽然牺牲了一定的性能和程序加载时间，但绝对是物有所值的。因为动态链接能够更有效的利用磁盘空间和内存资源，以极大方便了代码的更新和维护，更关键的是，它实现了二进制级别的代码复用。

库间依赖

不仅可执行程序会调用库，库也会调用其他库，库之间是有依赖的，那么如何做到库和库之间调用也是地址无关的呢？

因为库中也有**GOT**表，这就是为什么它们都是**ELF**格式文件。

总结

静态链接的出现，提高了程序的模块化水平。对于一个大的项目，不同的人可以独立地测试和开发自己的模块。通过静态链接，生成最终的可执行文件。

我们知道静态链接会将编译产生的所有目标文件，和用到的各种库合并成一个独立的可执行文件，其中我们会去修正模块间函数的跳转地址，也被叫做编译重定位(也叫做静态重定位)。

而动态链接实际上将链接的整个过程推迟到了程序加载的时候。比如我们去运行一个程序，操作系统会首先将程序的数据代码连同它用到的一系列动态库先加载到内存，其中每个动态库的加载地址都是不固定的，但是无论加载到什么地方，都要映射到进程对应的地址空间，然后通过**.GOT**方式进行调用(运行重定位，也叫做动态地址重定位)。